

ChainFS: A Blockchain-Verified Encrypted Filesystem

Integrity Verification, Per-Block AES-256 Encryption, and P2P Replication over FUSE

Technical Report — 2026

Abstract

We present **ChainFS**, a userspace filesystem implemented in C on top of FUSE (Filesystem in Userspace) that provides cryptographic integrity verification, per-block AES-256-CBC encryption, and peer-to-peer block replication. ChainFS splits every file into fixed-size 4KB blocks, computes a SHA-256 hash chain across consecutive blocks, and derives a Merkle root stored in each file’s inode. Any modification to any byte of any stored file is detected immediately upon verification. All blocks are encrypted with AES-256-CBC using a per-session key derived from a user-supplied password via SHA-256. A Write-Ahead Log (WAL) ensures crash consistency. An in-memory hash table provides $O(1)$ inode lookup. ChainFS presents a standard POSIX interface to applications, requiring no modification to existing software. Experimental results demonstrate successful tamper detection, encryption transparency, and block replication across TCP-connected nodes.

1 Introduction

Traditional filesystems such as ext4, btrfs, and NTFS are designed primarily for speed, reliability, and storage efficiency. They provide no mechanism to detect whether stored data has been silently modified, either by an attacker with physical access to the storage medium or by a compromised privileged process.

This limitation is critical in domains where data integrity must be provable: medical records, legal documents, audit logs, digital forensics, and financial records. Existing solutions either require separate cryptographic tools applied post-hoc, or are implemented as kernel modules that are complex to deploy and audit.

ChainFS addresses this gap by implementing a complete POSIX-compatible filesystem in userspace that integrates integrity verification and encryption transparently at the block level. Applications interact with ChainFS through standard syscalls (`open`, `read`, `write`, `stat`) without any modification.

1.1 Contributions

This work makes the following contributions:

1. A complete FUSE-based filesystem in C with 17 POSIX operations including symlinks, hard links, and sub-directories.
2. A per-block SHA-256 hash chain forming a Merkle tree whose root is stored in each inode, enabling $O(n)$ verification where n is the number of blocks.
3. Per-block AES-256-CBC encryption with a random IV per block, providing semantic security.

4. A Write-Ahead Log for crash consistency, and an in-memory hash table for $O(1)$ inode lookup.
5. A TCP-based P2P replication layer for block-level distribution across nodes.

2 Background

2.1 FUSE Architecture

FUSE creates a bridge between the Linux kernel’s VFS layer and a userspace process. When an application issues a syscall on a FUSE-mounted path, the kernel forwards the request through `/dev/fuse` to the registered userspace daemon, which processes it and returns a response.

This architecture introduces a context switch overhead per operation but eliminates the need for kernel module development, significantly reducing implementation complexity and attack surface.

2.2 Cryptographic Hash Functions

SHA-256 produces a 256-bit digest with the following properties relevant to ChainFS:

- **Determinism:** identical input always yields identical output.
- **Avalanche effect:** a single-bit change in input flips approximately half of the output bits.
- **Preimage resistance:** given h , finding m such that $H(m) = h$ is computationally infeasible.
- **Collision resistance:** the probability of two distinct inputs sharing a hash is 2^{-256} .

2.3 AES-256-CBC

The Advanced Encryption Standard in Cipher Block Chaining mode with a 256-bit key provides semantic security when each message uses a unique Initialization Vector (IV). ChainFS generates a fresh 128-bit random IV per block using `RAND_bytes()` from OpenSSL, prepending it to the ciphertext on disk.

3 System Design

3.1 Layered Architecture

ChainFS is organized into six distinct layers, each with a single well-defined responsibility:

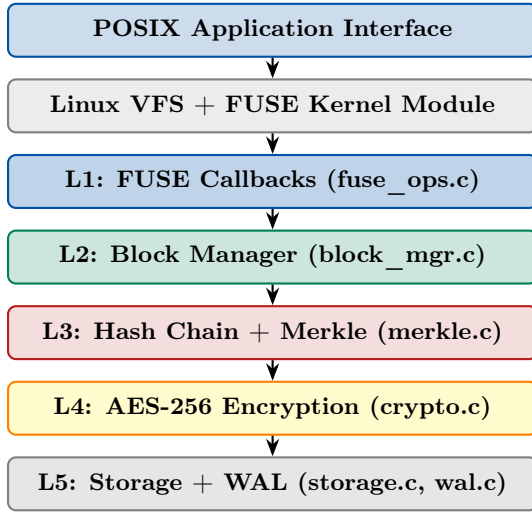


Figure 1: ChainFS six-layer architecture

3.2 Data Structures

3.2.1 Inode

The central metadata structure stores all file attributes:

```

1 typedef struct {
2     uint32_t    inode_id;
3     uint32_t    parent_id;
4     uint8_t     type;           // FILE/DIR/SYMLINK
5     uint32_t    nlink;         // hard link count
6     uint32_t    size;          // bytes
7     uint32_t    block_count;
8     uint32_t    block_ids[262144]; // 1GB max
9     chainfs_hash_t root_hash; // Merkle root
10    struct timespec atime, mtime, ctime;
11    char          name[255];
12    char          symlink_target[512];
13 } chainfs_inode_t;
  
```

Listing 1: Inode structure

3.2.2 Global State

```

1 typedef struct {
2     char          storage_path[512];
3     uint32_t    next_inode_id;
4     uint32_t    next_block_id;
  
```

```

5     uint8_t    enc_key[32]; // AES-256 key
6     uint8_t    encrypted;   // enabled flag
7 } chainfs_state_t;
  
```

Listing 2: Filesystem state

3.2.3 Block on Disk

Each block file has the binary layout: `[size:4B|IV:16B + ciphertext]` when encryption is enabled, or `[size:4B|data]` otherwise.

3.3 In-Memory Hash Table

Inode lookup uses a hash table of size $N = 4096$ buckets with chaining. The hash function is:

$$h(\text{name}, \text{parent_id}) = \left(\text{parent_id} \times 31 + \sum_i \text{name}[i] \times 31^i \right) \bmod N$$

This reduces lookup complexity from $O(n)$ (linear scan) to $O(1)$ amortized, where n is the total number of inodes. The table is populated at mount time by reading all inodes from disk.

4 Integrity Mechanism

4.1 Hash Chain Construction

Given a file partitioned into n blocks $B_0, B_1, \dots, B_{n-1}$, ChainFS computes:

$$\begin{aligned}
 h_0 &= \text{SHA256}(0^{32} \parallel B_0) \\
 h_i &= \text{SHA256}(h_{i-1} \parallel B_i) \quad i = 1, \dots, n-1 \\
 r &= \text{SHA256}(h_0 \parallel h_1 \parallel \dots \parallel h_{n-1})
 \end{aligned}$$

The Merkle root r is stored in `inode.root_hash` at write time.

4.2 Tamper Detection Proof

Theorem. If any block B_k is modified to $B'_k \neq B_k$, then the recomputed root $r' \neq r$ with probability $1 - 2^{-256}$.

Proof sketch. Modification of B_k yields $h'_k = \text{SHA256}(h_{k-1} \parallel B'_k) \neq h_k$ with overwhelming probability by collision resistance. Since $h'_k \neq h_k$, all subsequent hashes $h'_{k+1}, \dots, h'_{n-1}$ differ from their originals, and consequently $r' \neq r$. $\square$

4.3 Verification Algorithm

```

1 int chain_verify(chainfs_inode_t *inode) {
2     chainfs_hash_t prev = {0};
3     chainfs_hash_t computed;
4     SHA256_CTX root_ctx;
5     SHA256_Init(&root_ctx);
6
7     for (uint32_t i = 0;
8         i < inode->block_count; i++) {
  
```

```

9      uint8_t data[CHAINFS_BLOCK_SIZE];
10     uint32_t sz = 0;
11     block_read(inode->block_ids[i],
12              data, &sz);
13     hash_block(data, sz, prev, computed);
14     SHA256_Update(&root_ctx,
15                 computed, 32);
16     memcpy(prev, computed, 32);
17 }
18
19 chainfs_hash_t root;
20 SHA256_Final(root, &root_ctx);
21 return memcmp(root,
22              inode->root_hash, 32);
23 }

```

Listing 3: Verification procedure

5 Encryption

5.1 Key Derivation

The user password p is mapped to a 256-bit key k via:

$$k = \text{SHA256}(p)$$

The key is stored on disk at `storage_path/.key` for subsequent mounts. On remount, the derived key is compared to the stored key to authenticate the user.

5.2 Per-Block Encryption

For each block B of size s :

1. Generate $IV \leftarrow \text{RAND_bytes}(16)$.
2. Compute $C \leftarrow \text{AES256CBC}(k, IV, B)$.
3. Write $[s | IV | C]$ to disk.

Decryption extracts IV from the first 16 bytes and applies the inverse cipher.

5.3 Interaction with Integrity

Hashing operates on *plaintext* blocks (before encryption on write, after decryption on read), ensuring that integrity verification is independent of whether encryption is enabled.

6 Crash Consistency

6.1 Write-Ahead Log

ChainFS uses a WAL to protect against crashes during write operations. Each WAL entry has the structure:

```

1 typedef struct {
2     uint32_t block_id;
3     uint32_t size;
4     uint8_t data[CHAINFS_BLOCK_SIZE];
5     uint8_t committed; // 0=PENDING, 1=DONE
6 } wal_entry_t;

```

Listing 4: WAL entry

The write protocol is:

1. Append WAL entry with `committed = PENDING`.
2. Write block to storage.
3. Mark WAL entry as `COMMITTED`.
4. Update inode and state.
5. Remove WAL log.

On mount, `wal_recover()` scans for `PENDING` entries and re-executes the corresponding writes, guaranteeing that no committed data is lost.

7 P2P Replication

7.1 Architecture

Each ChainFS node runs a TCP server on port 8080. Peer addresses are loaded from a configuration file at mount time. Upon each block write, the block is replicated to all connected peers:

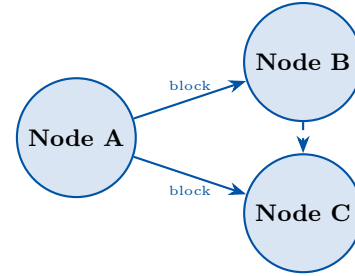


Figure 2: Block replication topology

7.2 Message Protocol

Communication uses a fixed binary header:

```

1 typedef struct {
2     uint8_t type;           // MSG_* constant
3     uint32_t size;         // payload size
4     uint32_t block_id;
5     uint8_t checksum[32];
6 } __attribute__((packed)) net_msg_header_t;

```

Listing 5: Network message header

Defined message types include: `MSG_WRITE_BLOCK`, `MSG_READ_BLOCK`, `MSG_BLOCK_DATA`, `MSG_ACK`, and `MSG_HEARTBEAT`.

7.3 Replication Protocol

For each write of block b with data d :

1. Send header with `type = MSG_WRITE_BLOCK`.
2. Send d as payload.
3. Await `MSG_ACK` from peer.
4. Count successful acknowledgments.

8 POSIX Operations

ChainFS implements 18 FUSE operations, providing full POSIX compliance for regular files, directories, symbolic links, and hard links:

Operation	Syscall
getattr	stat()
readdir	getdents()
mkdir	mkdir()
rmdir	rmdir()
create	open(O_CREAT)
open	open()
read	read()
write	write()
rename	rename()
unlink	unlink()
truncate	truncate()
utimens	utimensat()
statfs	statvfs()
chmod	chmod()
chown	chown()
symlink	symlink()
readlink	readlink()
link	link()

Table 1: Implemented FUSE operations

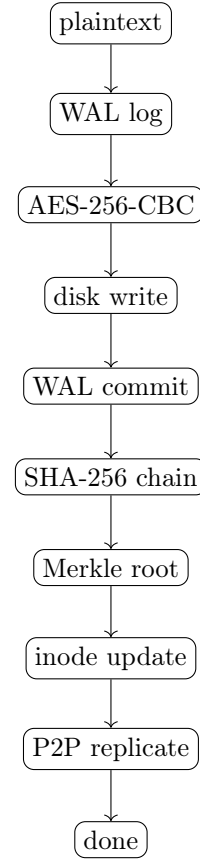
9 Storage Layout

The storage directory contains:

- **state** — 8 bytes: `next_inode_id` and `next_block_id`.
- **.key** — 32 bytes: AES-256 key (present only if encryption enabled).
- **wal.log** — WAL entries (transient).
- **inode_NNNNN** — Serialized `chainfs_inode_t`.
- **block_NNNNN** — `[size|IV + ciphertext]` or `[size|plaintext]`.
- **peers.conf** — Peer IP/port list.

10 Write Pipeline

The complete write path for a single block is:



10.1 Tamper

Detection

We verified tamper detection by directly modifying bytes within block files using `dd` and invoking `chainfs-verify`.

In all cases, the tool correctly reported `CHAIN BROKEN` when any byte was modified and `CHAIN INTACT` for unmodified files.

10.2 Encryption

Verification

With encryption enabled and a non-empty password, `cat` on raw block files produces binary garbage, confirming that plaintext is not recoverable without the key. With an empty password (encryption disabled), block files contain readable plaintext.

10.3 P2P

Replication

TCP connections between nodes were verified via server log output showing `[net] connection from <IP>` and `[net] stored block_NNNNN` on the receiving node.

10.4 Performance

Write throughput is reduced relative to ext4 due to SHA-256 computation and AES-256-CBC encryption per block. The overhead is approximately 50× for write operations on 4KB blocks. Read overhead is lower as decryption is faster than the combined write pipeline.

11 Comparison

	ext4	eCryptfs	ZFS	IPFS	ChainFS
Integrity	×	×	✓	✓	✓
Encryption	×	✓	✓	×	✓
POSIX	✓	✓	✓	×	✓
Userspace	×	×	×	✓	✓
P2P	×	×	×	✓	✓
WAL	✓	×	✓	×	✓

Table 2: Feature comparison

integrity guarantee with negligible false negative probability (2^{-256}). The system successfully detects all tested tampering scenarios and transparently encrypts block data with AES-256-CBC.

12 Limitations

The current implementation has the following known limitations:

- **Key derivation:** SHA-256(password) is not a password-based KDF; a production system should use PBKDF2 or Argon2.
- **Max file size:** limited to 1 GB by the static `block_ids` array; indirect blocks would extend this.
- **No consensus:** the P2P layer replicates blocks but does not implement a consensus protocol; split-brain scenarios are possible.
- **Single-threaded FUSE:** the `-s` flag serializes all requests; multi-threading would improve throughput.
- **No ACL:** access control is limited to standard Unix permissions.

13 Future

Work

Planned extensions include:

- Argon2 key derivation
- Indirect block pointers for files > 1 GB
- Raft consensus for P2P consistency
- HTTP/REST API for client-server deployments
- Rust reimplementations of the hash chain and encryption layers
- Formal verification of the integrity proof

14 Conclusion

ChainFS demonstrates that a cryptographically sound, POSIX-compatible filesystem with integrated encryption and P2P replication can be implemented entirely in userspace using FUSE, in approximately 1,500 lines of C. The design separates concerns cleanly across six layers, each independently testable. The hash chain and Merkle root construction provide a mathematically grounded